

Chapter 4

Exception Handling

1. Introduction

- An exception is an abnormal condition that arises at run time.
 - ✓ Is a run-time error
- With out support of exception handling, errors must be checked and handled manually
 - ✓ typically through the use of error codes, and so on.
- Java's exception handling brings run-time error management into the object-oriented world.
 - ✓ A Java exception is an object that describes an exceptional (error) condition that has occurred in a piece of code.

Introduction

- When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error.
 - ✓ That method may choose to handle the exception itself, or pass it on.
 - ◆ Either way, at some point, the exception is caught and processed
- Exceptions can be generated by the Java run-time system, or by your code manually.
 - ✓ Exceptions thrown by Java relate to fundamental errors that violate the rules of the Java language or the constraints of the Java execution environment.
 - ✓ Manually generated exceptions are typically used to report some error condition to the caller of a method.
- Java exception handling is managed via five keywords: try, catch, throw, throws, and finally

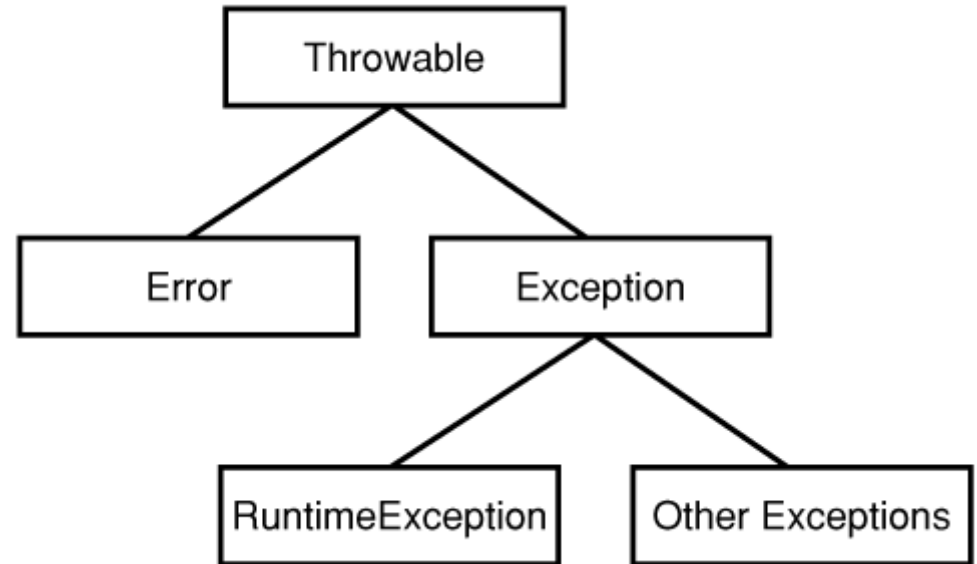
1. Introduction

- This is the general form of an exception handling block.

```
try {  
    // block of code to monitor for errors  
}  
catch (ExceptionType1 exOb) {  
    // exception handler for ExceptionType1  
}  
catch (ExceptionType2 exOb) {  
    // exception handler for ExceptionType2  
}  
// ...  
finally {  
    // block of code to be executed after try block ends  
}
```

2. Exception Types

- All exception types are subclasses of the built-in class Throwable.
- Throwable has two subclasses, Exception and Error.
- Error
 - ✓ Represent abnormal situations that happen in the JVM.
 - ✓ Cannot usually be handled by your program.
 - ✓ Are used by the Java run-time system to indicate errors related to run-time environment



2. Exception Types

➤ Exception

- ✓ Is an unwanted event in a program like invalid array indexing.
- ✓ Can be handled within the program
- ✓ Extend it to create custom exception types
 - ◆ All Java exception classes inherit directly or indirectly from class Exception
- ✓ Has subclass RuntimeException

➤ RuntimeException

- ✓ Automatically defined for the programs that you write
- ✓ Include things such as division by zero and invalid array indexing.

3. Uncaught Exception

- Is unhandled exception that would terminate the program.
- This can lead to unexpected behavior and may cause a loss of data.

```
1 public class Exceptions {  
2     public static void main(String[] args) {  
3         divide();  
4         System.out.println("Program Exits Normally");  
5     }  
6     static void divide(){  
7         int n = 8, d = 0;  
8         System.out.println(n/d);  
9     }  
}
```

- The above program displays the following and terminates.

Exception in thread "main" java.lang.ArithmeticException: / by zero

3. Uncaught Exception

- If we rewrite the main method as follows.
- ```
1 public static void main(String[] args) {
2 try{
3 divide();
4 }
5 catch(ArithmeticException aexp){
6 //do something to handle the exception here
7 }
8 System.out.println("Program Exits Normally");
9 }
```
- Then the program execute the code in the catch block and prints "Program Exits Normally" message.
  - ✓ The program doesn't terminate



## 4. Using try and catch

- Java run-time system provides default exception handler.
- However, handling an exception yourself provides two benefits.
  - ✓ Allows you to fix the error.
  - ✓ Prevents the program from automatically terminating.
- To handle a run-time error, enclose the code that you want to monitor inside a try block.
- One or more catch block immediately follows the try block
- catch specifies the exception type that you wish to catch.
- Once an exception is thrown, program control transfers out of the try block into the catch block.
- A try and its catch statement form a unit.

## 5. Multiple catch Clauses

- More than one exception could be raised by a single piece of code.
- We can specify two or more catch clauses, each catching a different type of exception.
  - ✓ Each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
  - ✓ Others are bypassed, and execution continues after the try / catch block
    - ◆ You should place catch blocks for more specific exception types before those for more general types
- A single catch block can handle more than one type of exception.
  - ✓ The catch clause specifies the types of exceptions that the block can handle, and each exception type is separated with a vertical bar (|).
    - ◆ the catch parameter is implicitly final.

## 6. Nested try Statement

- The **try** statement can be nested.
- Each time a **try** statement is entered, the context of that exception is pushed on the stack.
- If an inner **try** statement does not have a **catch** handler for a particular exception, then the **catch** blocks of parent **try** block are inspected for that exception
- This continues until one of the catch statements succeeds, or until all of the nested try statements are exhausted.
- If no catch statement matches, then the Java run-time system will handle the exception

## 7. Using throw

- Exceptions can be thrown by the Java run-time system.
- Your program can also throw an exception explicitly, using the **throw** statement.

**throw** ThrowableInstance;

- ThrowableInstance is an object of type Throwable or a subclass of Throwable.
  - ✓ Can be obtained by using a parameter in a catch clause or creating one with the new operator.
- The flow of execution stops immediately after the throw statement
  - ✓ Controls transferred to the matching catch block
    - ◆ Handled by default exception handler If no matching catch is found.

## 8. Using throws

- **throws** is used in the signature of a method to indicate that this method might throw one of the listed exception types.
- The caller to these methods has to handle the exception.
- Its general form is:  

```
type method_name(parameters) throws exception_list { ... }
```

  - ✓ **Exception-list:** comma-separated list of exceptions that a method can throw.
- Required only for checked exceptions
  - ✓ its usage is meaningless for unchecked exceptions.

## 9. finally Block

- *finally* creates a block of code that will be executed after a try /catch block has completed
  - ✓ and before the code following the try/catch block or.
  - ✓ just before the method returns if the method returns from inside a try/catch block via an uncaught exception or an explicit return statement.
- The finally block will execute whether or not an exception is thrown.
- It is useful for freeing up resources that might have been allocated at the beginning of a method.
- The finally clause is optional. However, each try statement requires at least one catch or a finally clause.

# 10. Java Built-in Exceptions

- Exceptions can be categorized into built-in and user defined
- Built-in exceptions
  - ✓ Exceptions that are already available in Java libraries.
  - ✓ can be categorized into checked exceptions and unchecked exception.
- Checked Exception
  - ✓ Are called compile-time exceptions
  - ✓ checked at compile-time by the compiler.
    - ◆ The compiler ensures whether the programmer handles the exception or not.
    - ◆ have to be handled ; otherwise, the system show a compilation error.
  - ✓ handle checked exception by using try catch block or by using the throws keyword

# 10. Java Built-in Exceptions

## ➤ Unchecked Exceptions

- ✓ The compiler will not check these exceptions at compile time.
- ✓ No compile time error even if we didn't handle or throws it.
- ✓ Usually, it occurs when the user provides bad input.
- ✓ Most of them are subclasses of `RuntimeException` inside `java.lang` package



# 11. Creating your own Exception

- To create your own exception, define a subclass of class `Exception`
  - ✓ No need to implement anything
    - ◆ It's existence allows us to use them as exceptions.
  - ✓ The `Exception` class does not define any methods of its own.
    - ◆ All of it's methods are inherited from `Throwable`.
- `Throwable` defines
  - ✓ Constructors `Exception( )` and `Exception(String msg)`
  - ✓ And overrides the `Object` method `toString()`
- You can specify the description of your exception either
  - ✓ By using the constructor with `String` parameter
  - ✓ or by overriding `toString()` method

## 12. try-with-resources

- Automates the process of releasing a resource, such as a file, when it is no longer needed.
- It prevents situations in which a file (or other resource) is inadvertently not released after it is no longer needed.
- Its general form is: **try** (resource-specification) {*// use the resource* }
- Resource-specification is a statement that declares and initializes a resource, such as a file stream.

```
try(FileInputStream fin = new FileInputStream(fileName)) { ... }
```

- The stream associated with fin is automatically closed when the try is left
- The variable fin is local to the try block and is implicitly final.